FastAPI Conf, Amsterdam / October 2026

Constrained Optimism

Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic** (the validation layer under FastAPI), Founder & CEO of **Pydantic Labs**

Pydantic Today

Open source

FREE – WHAT YOU BUILD WITH

Pydantic Validation
Contracts for data, 1 billion downloads a month

Pydantic AI
Production grade agent framework

Pydantic Monty
Python sandbox for code written by agents, starts in 1ms

Pydantic Logfire

COMMERCIAL PLATFORM

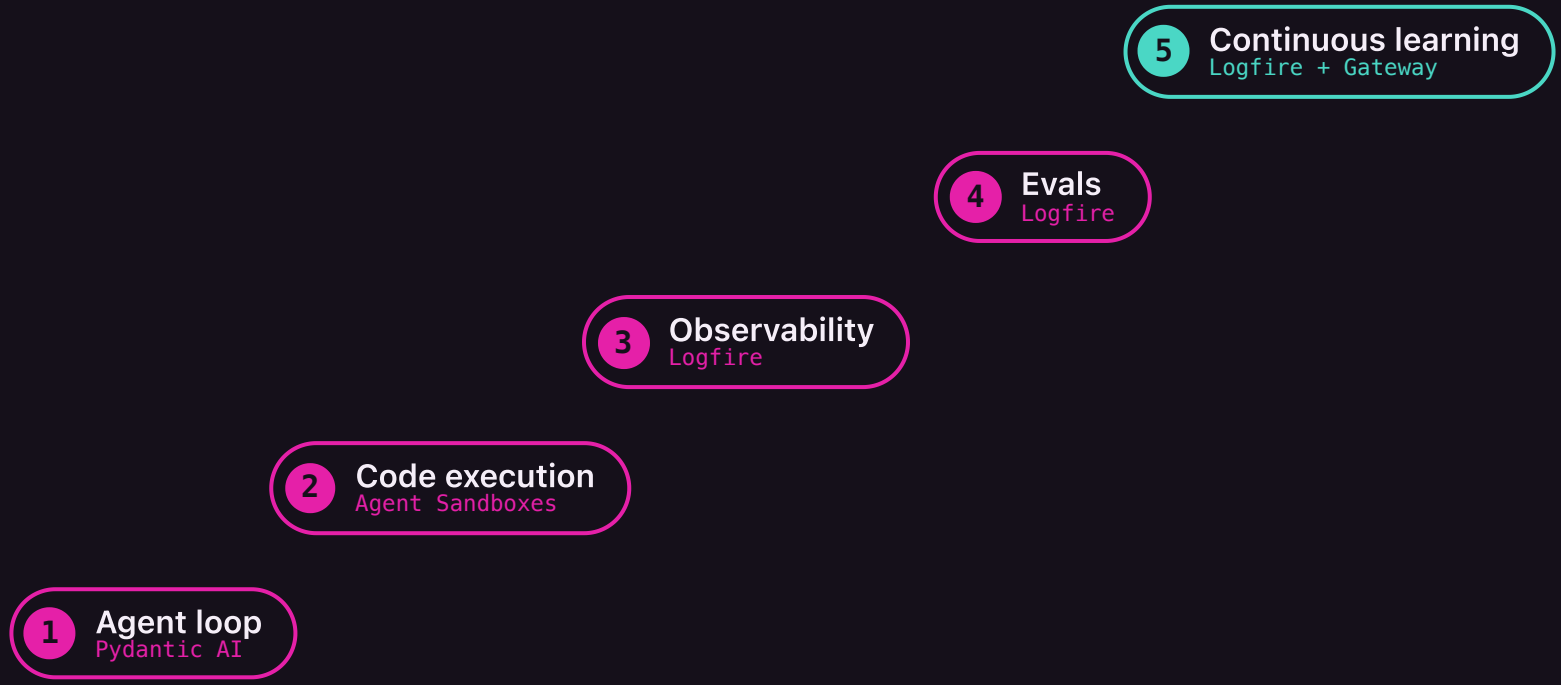
AI Observability
Traces, evals and prompt management for agents

Infra Observability
Logs, metrics and traces for the whole application

AI Gateway
One endpoint, every model

Agent Sandboxes soon
Hosted code execution, built on Monty

What Pydantic does, in five steps



That's the day job. This talk is about something else.

How I think about building software now.

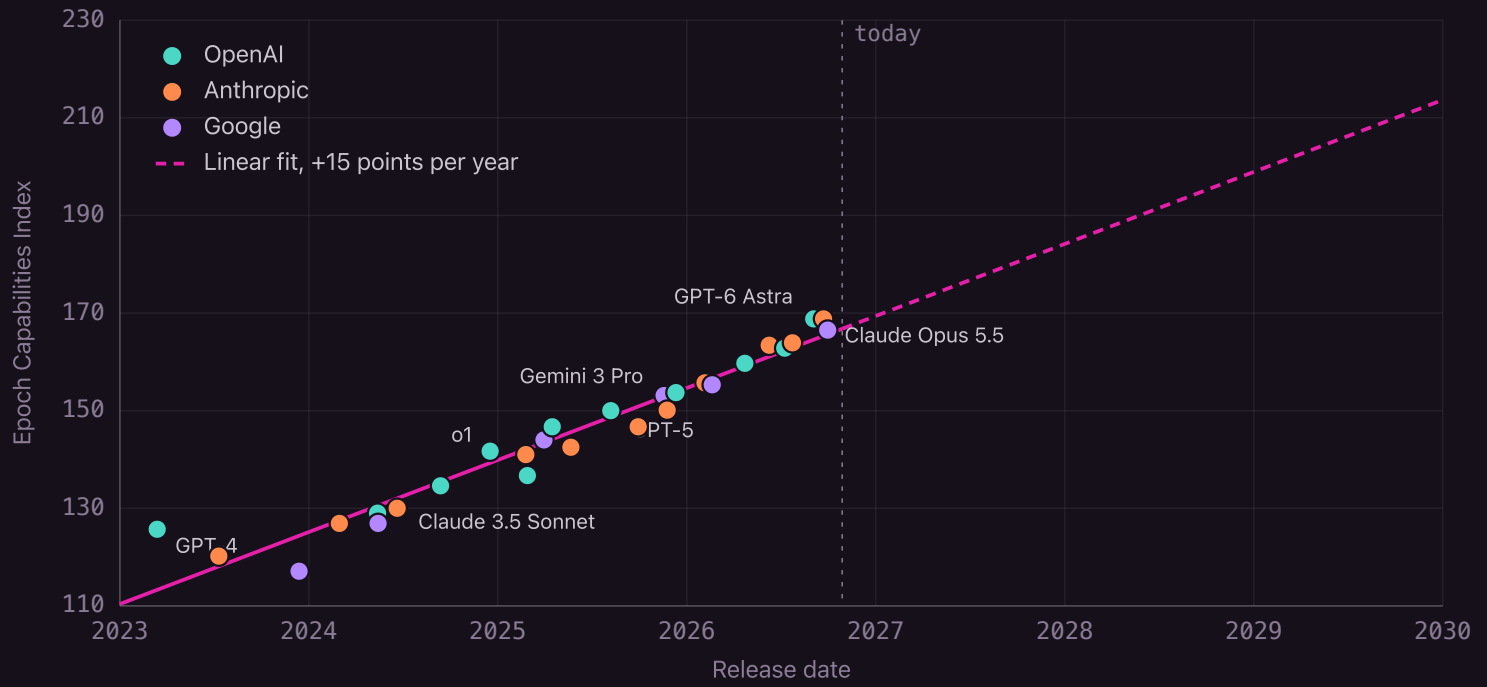
It's a theory. I'm not certain it's right today, let alone tomorrow.

Aside

Will the models keep getting smarter?

Model intelligence over time

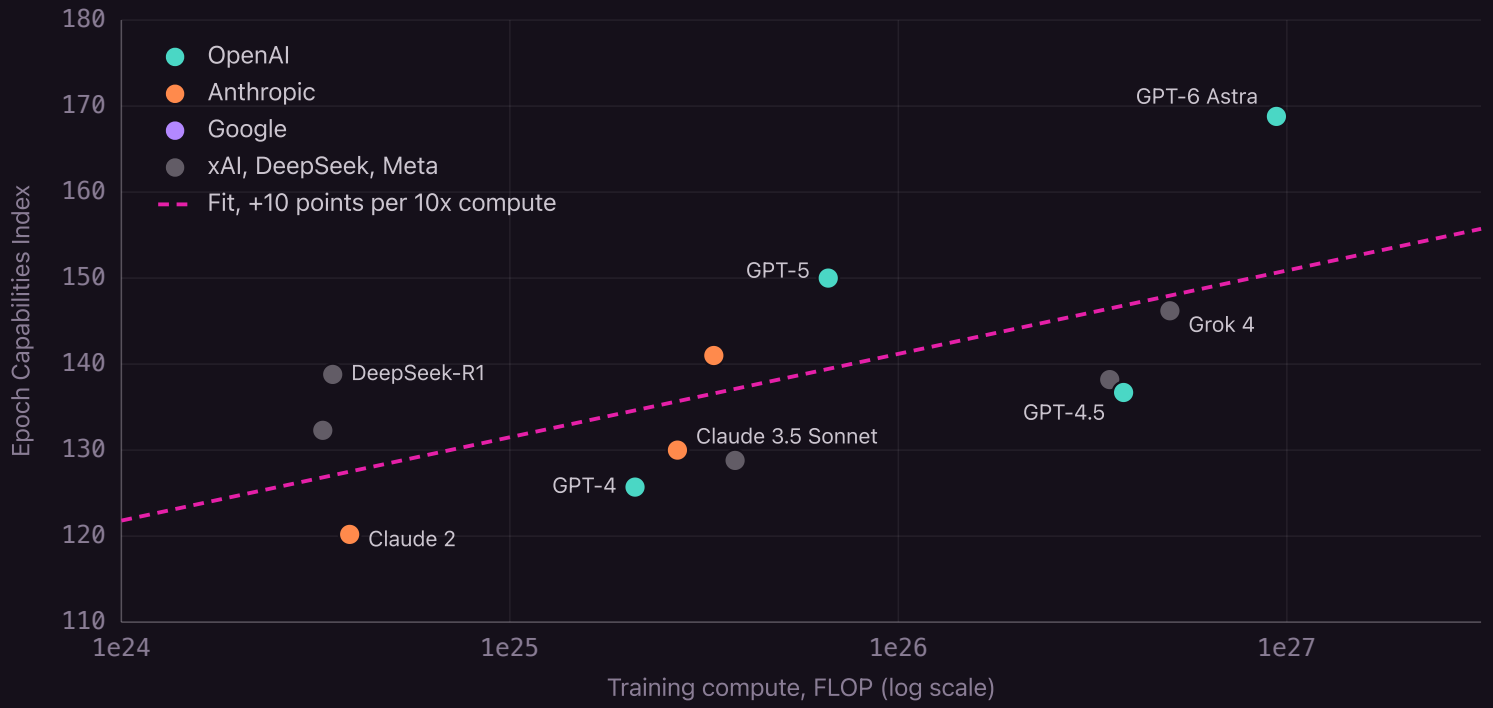
A purely empirical relationship!



Sources on GitHub

Model intelligence vs. training compute

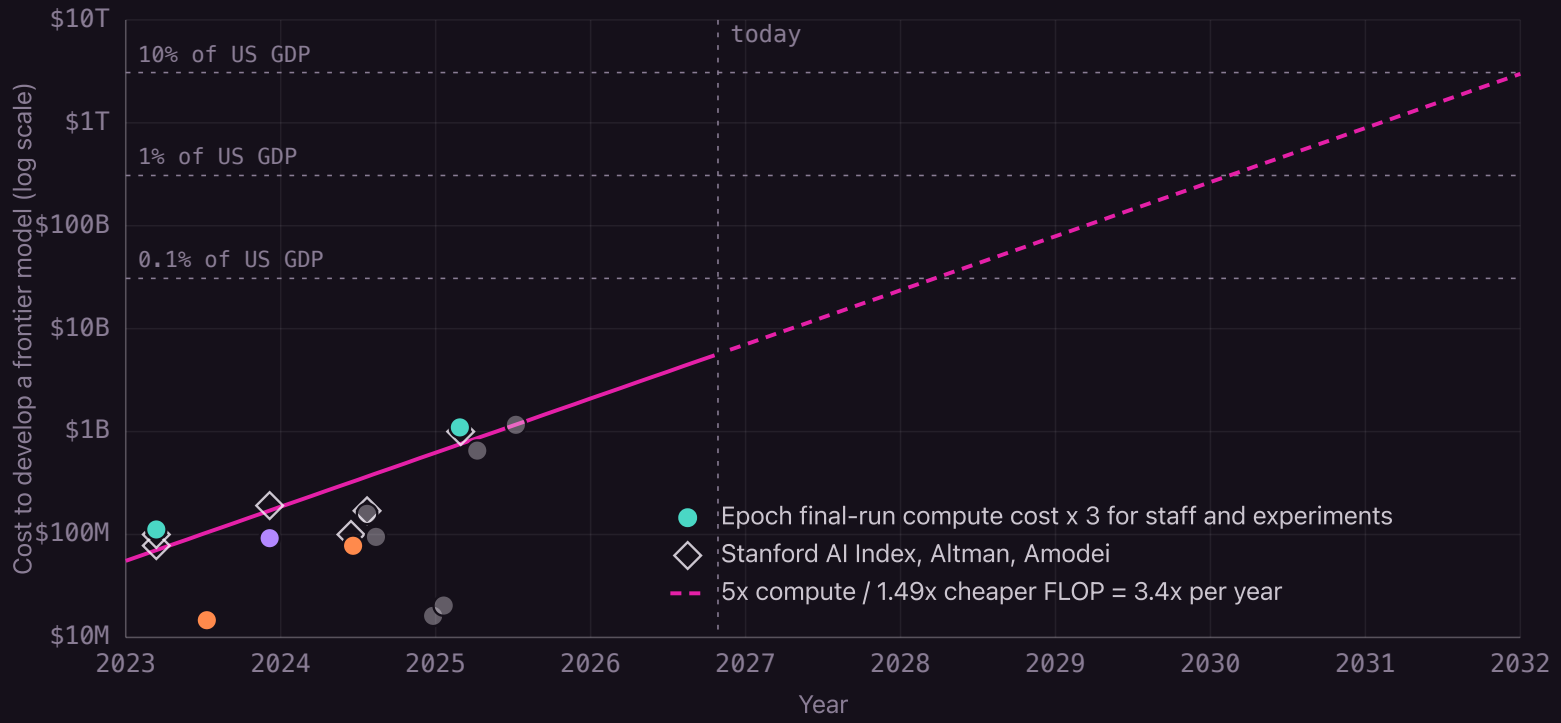
Another empirical relationship - Intelligence rises with the log of training compute



Sources on GitHub

What it costs to stay on the line

If intelligence is to continue increasing linearly *(all other things being equal)*, the cost will increase exponentially.



Sources on GitHub

Summary

- We have **no mechanistic model** that says models will keep getting more intelligent
- Indeed, we have a model that says they **won't**
- **But**, in the past, the tech industry found ingenious ways (to keep Moore's law going)

It's dumb to assume models will keep getting more intelligent, and dumb to assume they won't!

What does this mean for Software Developers?

Software development is a constrained, non-convex optimization problem.

What does that mean?

> *Software development is a constrained, non-convex optimization problem.*

Optimization

There are metrics and you want them better: speed, compatibility, memory, signups.

The agent is the optimizer. Stochastic, tireless, not picky about how.

Constrained

Some things must stay true whatever the metric says: the spec, security, no `unsafe`.

Gates, not weights. The optimizer never gets to trade them.

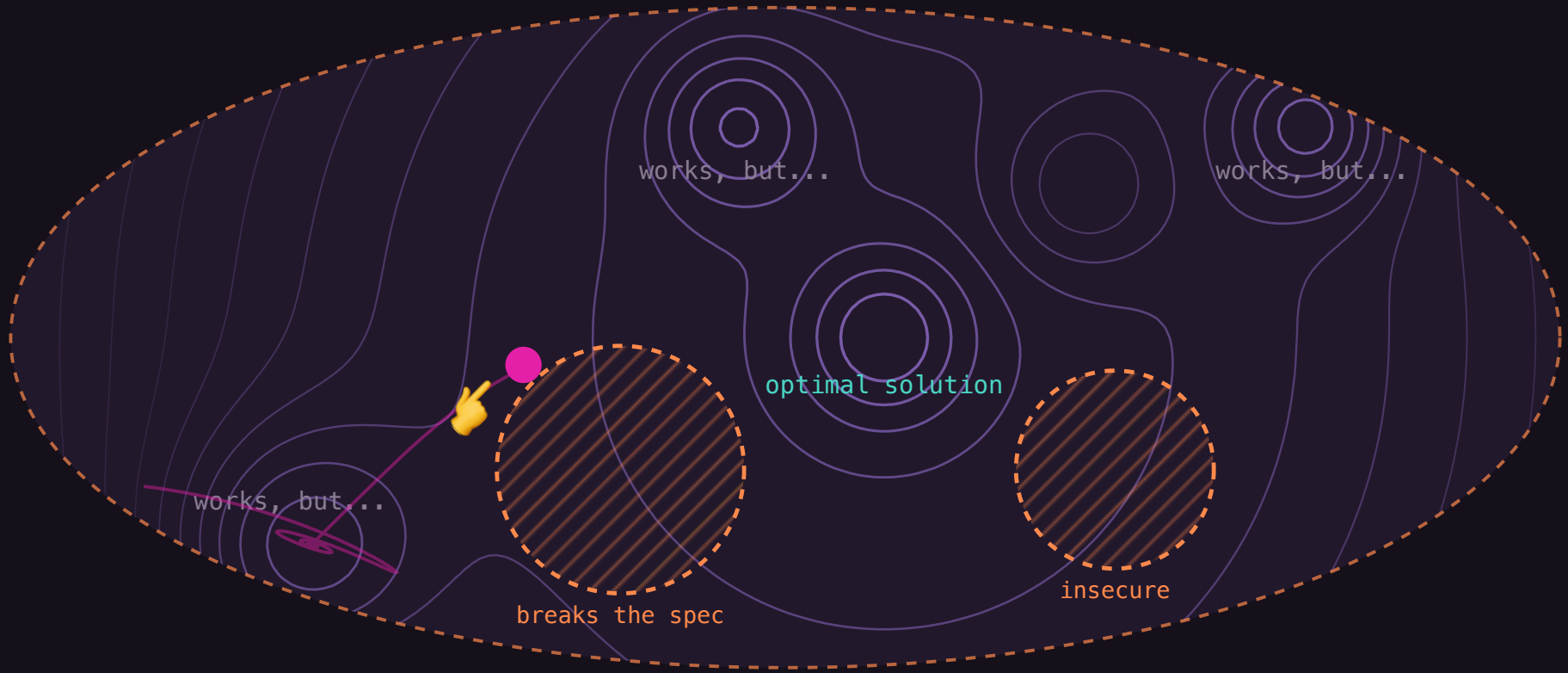
Non-convex

There is no single downhill path. Plenty of places "work" but aren't good, and the optimizer will stop in one.

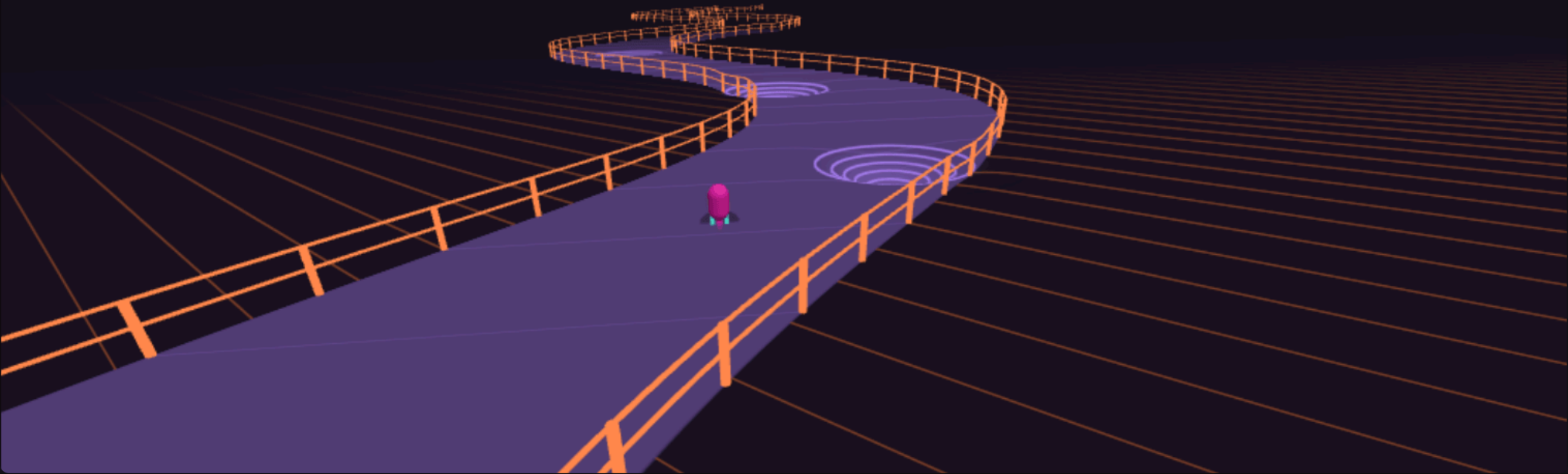
Someone has to poke it.

My job moved from finding **x** to choosing **f** and **g**. Review is one of the **g**'s.

Visual slop metaphor



But we never reach perfect, the descent is indefinite



So what does the developer actually do?

Set up the problem for the agent to solve.

The challenge

1

Pick a few metrics

What gets better. If there seem to be ten, combine them into one composite score.

`p50 latency` `CPython compat %` `sign-up success %`

2

Define hard constraints

What must never break. Deterministic where you can, AI review where you can't.

`cargo test` `pyright --strict` `clippy -D warnings` `review agent`

3

Close the feedback loop

The agent reads the result itself: no human copying errors between windows. Human review is still important.

`gh run view` `compiler errors` `CodSpeed MCP` `Logfire MCP`

THIS IS NOT A SOFTWARE FACTORY!

How this feels

```
def implementation(a: int, b: str, c: float) -> Foo:
    return foo
```



Monty

A Python interpreter in Rust for running the code agents write. **Starts in under 1 ms.**

Optimize

CPython compatibility

Startup and runtime speed

Memory

Constraints

No way out of the sandbox

Resource limits always hold

No `unsafe`

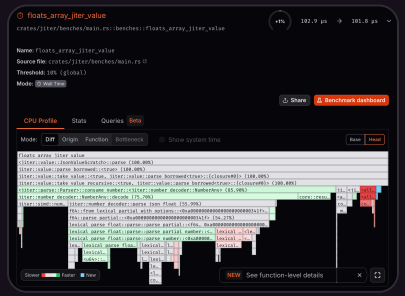
Code readability

Monty: how the agent gets feedback

Metrics

```
x = {}
x.get(1, 2, 3)
# Raise=TypeError('get expected at most 2 arguments, got 3')
```

- **~700 Python scripts** run with Monty and CPython, guaranteeing exactly matching behaviour
- **CodSpeed** on every PR, with an MCP server so the agent reads the result itself. Don't be a meat proxy.

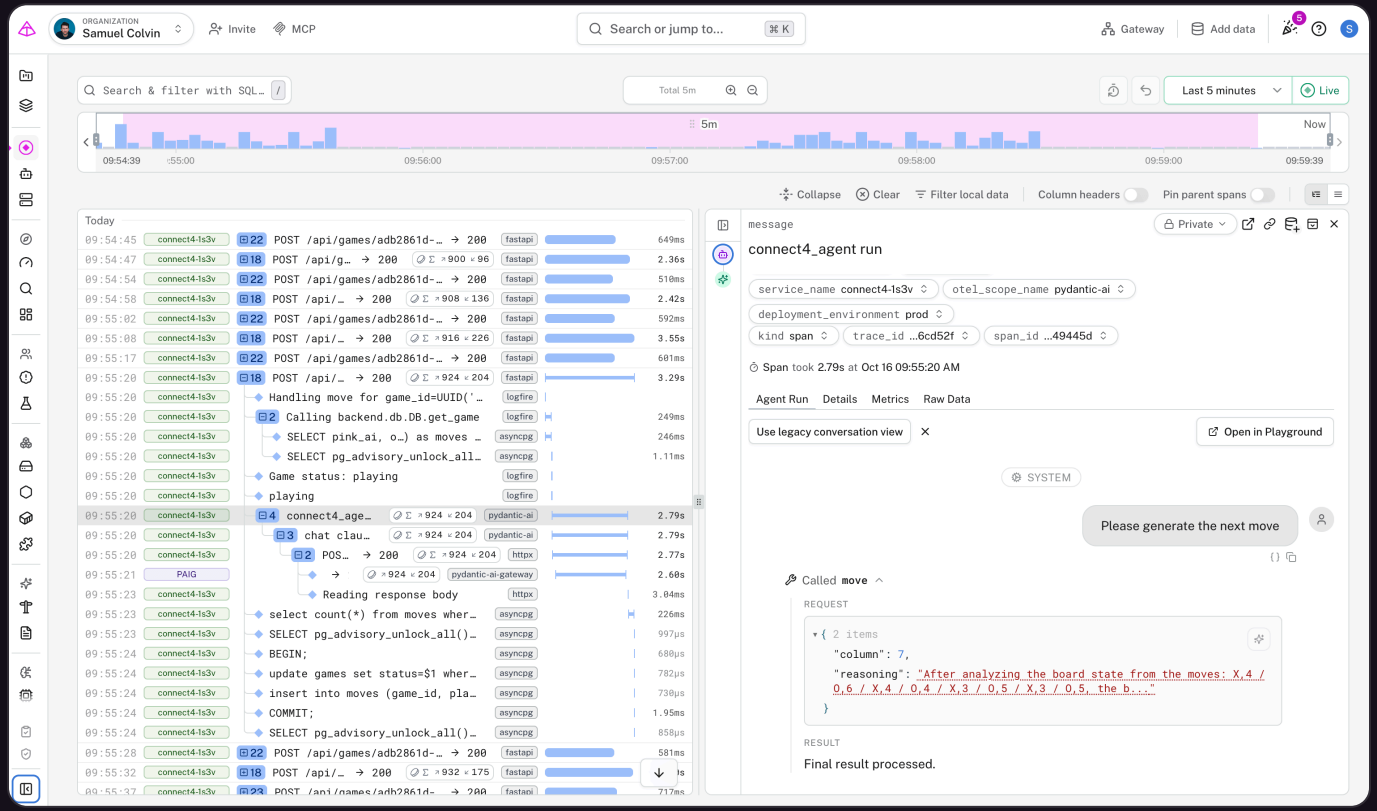


Constraints

- **Masses of unit tests**, under `cargo careful` and **miri**, which catch undefined behaviour
- `unsafe_code = "deny"`
- **Clippy**, pedantic group denied
- **Fuzzing** against adversarial input
- **Reference-counting build**
- **Python tests** on every supported version
- **Builds** on the minimum Rust version and on the latest dependencies
- **Bounty program**: hackmonty.com, $20k reward
- **3 review agents**: Cubic, Macroscope, Veria Labs

Logfire

AI + Infra observability, evals, AI Gateway



Logfire

AI + Infra observability, evals, AI Gateway

Optimize

Business: a Pareto frontier across signup success, engagement, new feature usage and data sent

Engineering: query performance, ingest latency

Constraints

Tenant isolation

Auth security

Alerts must fire

Ingest uptime

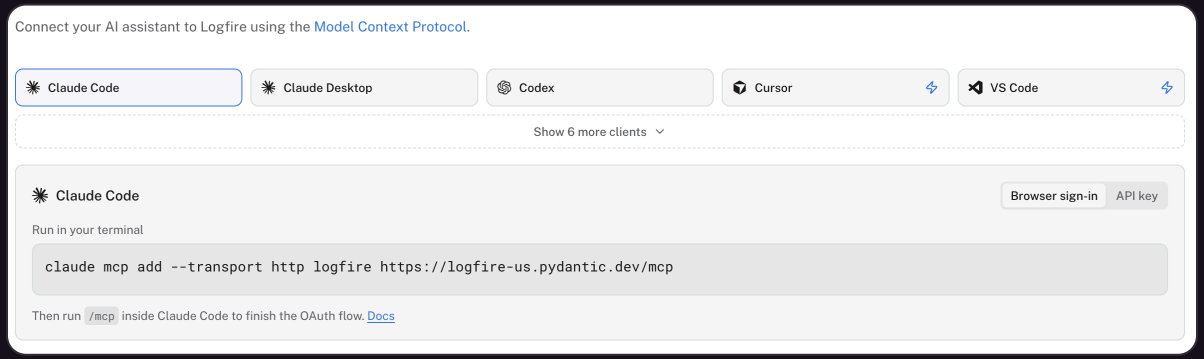
Logfire Example

Our MCP server lets coding agents query Logfire directly with SQL and do (almost) **everything**. This lets our customers use Logfire as a metric or a constraint.

The constraints are obvious (security, isolation). The metrics are harder to define:

- Speed to achieve a goal (we can't measure token count)
- Valid SQL %

We get feedback by including an `intent` field in MCP tools.



Where the metaphor breaks

- **Constraints are sampled, not proven.** CI checks points, not the region. Types and proofs are the exception, and the main reason to stay strict about typing.
- **The optimizer can read the constraints.** A solver can't lobby you to relax a bound; an agent will edit the test. Fences need tamper-protection: branch protection, "never xfail", a human approval, credentials it doesn't have.
- **The real objective is a proxy.** Signups, revenue, "ease of use" aren't measurable until deployed. Goodhart's law applies to you, not just the model.

Why I'm optimistic

Defining and maintaining the fences on an endless slope is a harder, more complex and more valuable job than typing out the code ever was.

Thank you



github.com/pydantic/talks/tree/main/2026-10-28-fastapiconf

`@samuelcolvin` · `@pydantic`